EXPERIMENT NO: 6

Student Name:DIVYANSH **UID:** 23BCS11778

Branch: BE-CSE **Section/Group:** KRG-2A

Semester: 6th **Date of Performance:** 03/01/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

To design a **scalable video streaming system similar to YouTube** that supports **user onboarding, video upload, processing, search, playback, likes, comments, and subscriptions**, while ensuring **high availability, low latency, and fault tolerance** using a distributed system architecture..

Objectives

- Understand the **high-level architecture** of a large-scale video streaming platform
- Identify **functional requirements** such as video upload, streaming, subscriptions, likes, and comments
- Identify **non-functional requirements** like scalability, availability, and performance
- Analyze **CAP theorem trade-offs** in different components of the system
- Design **RESTful APIs** for user management, video handling, and engagement feature.

Procedure

- Studied real-world video platforms such as YouTube
- Identified major system components shown in the HLD diagram
- Analyzed complete user flow:
  - User registration
  - Video upload
  - Video processing
  - Content delivery via CDN
- Defined functional and non-functional requirements
- Designed APIs corresponding to each service
- Analyzed scalability for millions of concurrent viewers
- Studied consistency vs availability trade-offs for views, likes, and comments

Functional Requirements

- Users should be able to **sign up and log in securely** to the platform.
- Users should be able to **upload videos** to the platform.
- Users should be able to **stream/watch videos** without downloading them.

- Users should be able to **watch videos in multiple quality levels** (e.g., 144p, 360p, 720p, 1080p).
- Users should be able to **search videos** using **title, tags, or channel name**.
- Users should be able to **like videos, comment on videos, and subscribe to channels**.
- Users should be able to **purchase and manage subscriptions** (premium or channel subscriptions, if applicable).

Core Entities of the System

1. • CLIENTS
2. CLIENTS METADATA
3. VIDEOS
4. VEDIOS META DATA

API Design

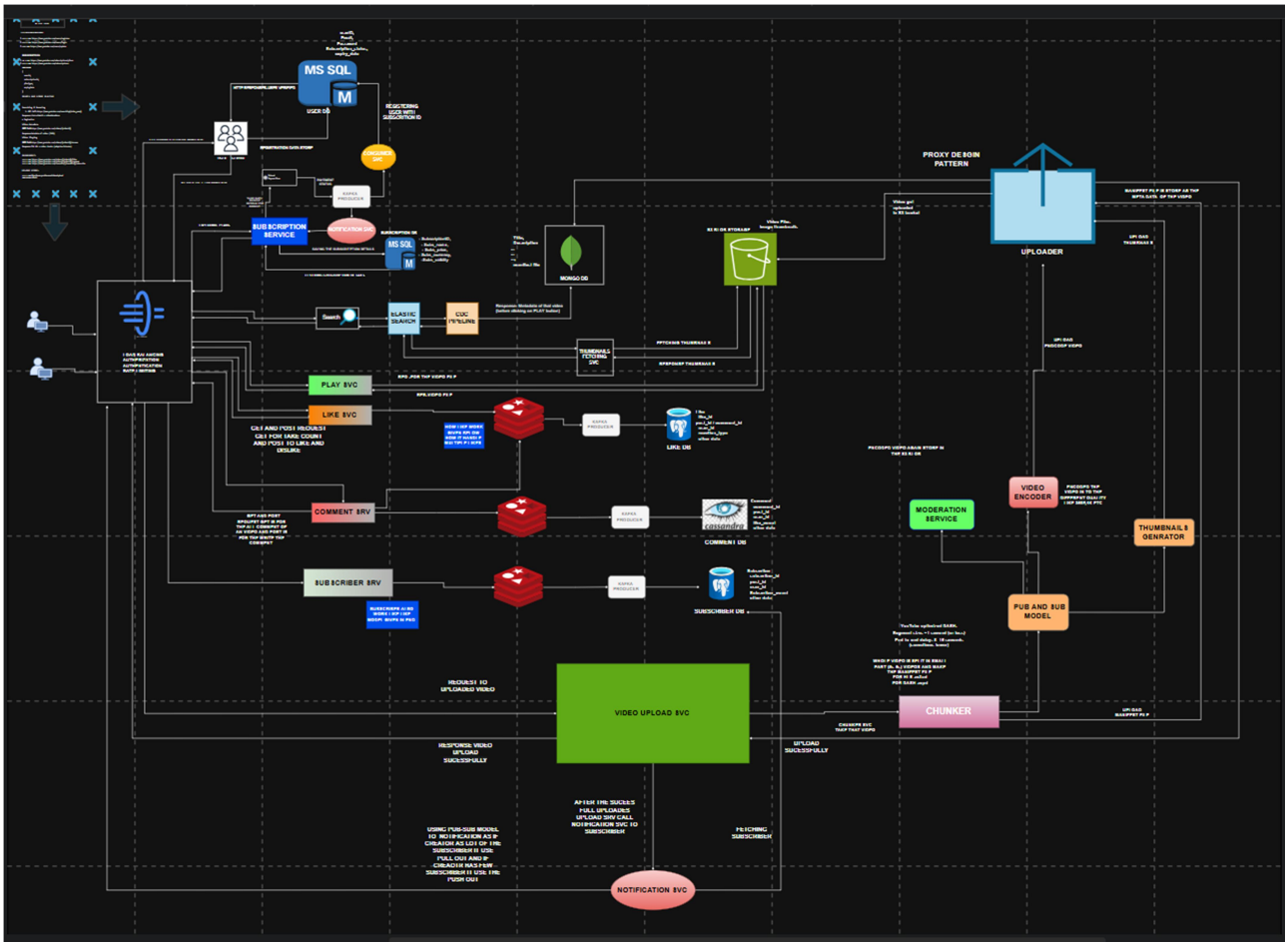
- **User Registration** – POST /user/register
- **User Login** – POST /user/login
- **Upload Video** – POST /video/upload
- **Stream Video** – GET /video/stream/{video_id}
- **Search Videos** – GET /video/search
- **Like Video** – POST /video/like
- **Comment on Video** – POST /video/comment
- **Subscribe Channel** – POST /channel/subscribe

Non-Functional Requirements

1. SCALBILITY 200M-400M USER FOR WHICH WE HAVE 50K VIDEOS
2. LATENCY 50MS-100MS
3. AVAILABILITY>>>>> CONSISITENCY FOR OVERALL
4. CONSISTENCY>> AVAILABILITY FOR SUBSCRIPUTION
5. CILENT SHOULD ABLE TO SEE THE VIDEO ZERO OR MINIMAL BUFFERING .

High Level Design (HLD)

Users can sign up and log in securely to the platform. They can upload videos and stream videos online without downloading. The system allows users to watch videos in multiple quality levels such as 360p, 720p, and 1080p. Users can search videos using titles, tags, or channel names. They can also like videos, comment on them, subscribe to channels, and take platform subscriptions if available.



Outcome

- Designed a **YouTube-like scalable video streaming system**
- Identified **functional and non-functional requirements**
- Designed **REST APIs** for core features
- Understood **scalability challenges** in video platforms
- Analyzed **CAP theorem trade-offs**
- Successfully mapped system design **according to the HLD diagram**

.